

Crop-Centered Agricultural Advisory System

System Architecture & Conception Document — v3.0

Field	Value
Project	Crop-Centered Agricultural Advisory System
Version	3.0 — Crop-Centered Architecture Baseline (supersedes v2.0, region-centered)
Status	Final Conception / Implementation Baseline
Author	MEDAH MAFFO LAPNET LELICA + Architecture Specification
Engine	Python
Frontend	Flutter
Pilot	West Region, Cameroon (first crop catalogue)
Orientation	Crop-centered advisory (was region-centered in v2.0)
Decision model	Interconnected rule-based decision forest, rooted on the target crop
Channels	Mobile + SMS + Voice/Audio
Standards framing	IEEE 1016-style + C4 + UML
Roadmap	V1 Advisory → V2 Cultivation Assistant → V3 IoT → V4 Multi-Sector

Table of Contents

Table of Contents	2
1. Document Control and Architecture Principles.....	4
1.1 Executive Summary.....	4
1.2 Why Crop-Centered Instead of Region-Centered.....	4
1.3 Architectural Separation (unchanged principle).....	4
1.4 The Crop Passport — A Unifying, Lifecycle-Centric Concept.....	5
2. Scope, Stakeholders and Requirements	5
2.1 Stakeholders	5
2.2 Functional Requirements — V1 Crop-Centered Advisory.....	5
2.3 Non-Functional Requirements.....	6
3. UML Use-Case Model.....	7
4. C4 Architecture Views.....	8
4.1 C4 Level 1 — System Context	8
4.2 C4 Level 2 — Container View.....	8
5. Crop-Centered Knowledge Forest — Seven Working Trees.....	8
5.1 Worked Example — West Region Baseline, Read Crop-First	9
5.2 Soil Suitability & Improvement — What Changes	9
5.3 Soil, Climate, Timing and Practices (carried over, now crop-qualified)	10
6. Domain Model — Entities and Relationships.....	10
7. UML Component Model — Developer 1.....	11
8. Activity Model — Engine Execution.....	11
9. Sequence Models.....	12
9.1 Mobile Interactive Advisory	12
9.2 Proactive SMS Advisory.....	12
9.3 External Weather Failure.....	13
10. Scoring, Constraints, Ranking and Conflict Resolution	13
10.1 Mobile Score.....	13
10.2 SMS Priority	13
10.3 Governance.....	14
10.4 Conflict Resolution	14
10.5 Structured Output.....	14
11. Interface Contracts and API Design	14
11.1 API Endpoints.....	15
11.2 Canonical Recommendation Contract	15
12. Data Architecture.....	15

12.1 Data Governance	15
13. UML Deployment and Security Architecture.....	15
13.1 Security Controls.....	16
13.2 Deployment Decision	16
14. Python Repository and Developer Work Split.....	16
14.1 Responsibility Matrix.....	17
15. Traceability, Explainability and Past/Present/Future	17
15.1 Past.....	18
15.2 Present	18
15.3 Future.....	18
16. SMS Channel Architecture and Simulation	18
16.1 What the Simulator Needs	18
16.2 SMS Content Priorities	18
17. Testing, Validation and Risk Management.....	19
17.1 Example Scenario.....	19
17.2 Risk Register.....	19
18. Evolution Roadmap.....	20
18.1 V1 — Crop-Centered Advisory (this document's baseline)	20
18.2 V2 — Cultivation Assistant	20
18.2.1 Lifecycle Guidance.....	20
18.2.2 Real-Time Information Reliance.....	21
18.2.3 Notification Engine	21
18.2.4 Plant Disease/Pest Detection and Response	21
18.2.5 V2 Component and Data Impact.....	21
18.3 V3 — IoT Soil Sensor Integration	22
18.4 V4 — Multi-Sector Expansion	22
18.5 Roadmap Summary Table.....	22
18.6 Open Decisions by Stage	23
19. Architectural Acceptance Baseline	23
19.1 Source Basis and Traceability	24
19.2 Professional Deliverables Included	24

1. Document Control and Architecture Principles

Item	Decision
Primary scope	V1 system architecture and technical conception, crop-centered
Orientation change	The system is re-rooted on the target crop; region, soil, topography, climate, timing and practices become supporting context evaluated against that crop
Engine	Python rule-based inference engine (unchanged)
Client	Flutter mobile application (unchanged)
V1 hardware	None / no IoT (IoT sensing arrives at V3, see Section 18)
Knowledge	Seven interconnected trees, rooted on the Crop Profile tree
Decision separation	WHAT = crop knowledge forest; HOW = inference engine
External services	Weather, translation, TTS, SMS via interfaces/adapters

1.1 Executive Summary

The platform provides crop-oriented agricultural advice: a farmer selects or describes the crop they grow or intend to grow, and the system reasons about how to make the land, the soil and the practices suitable for that crop, and how to improve yield for it. Advice ranges from soil improvement actions to full crop-suitability assessments, still delivered in advisory mode — the farmer asks, the system answers with a traceable recommendation. The same reasoning core continues to support two channels: Mobile is interactive and answers a farmer's crop question; SMS is proactive, sending crop-calendar and risk information to farmers who do not use the application. Voice/audio remains a delivery path after translation and text-to-speech. The mobile application also keeps its education section, and the SMS channel keeps its informative, app-free character.

1.2 Why Crop-Centered Instead of Region-Centered

Version 2.0 entered every recommendation from the region: a farmer's location fixed the disease profile, the topography options and the crop candidates offered. Version 3.0 inverts the entry point. The farmer's question is now built around a crop — "I grow potatoes, how do I improve my yield" or "I want to plant coffee, is my land suitable" — and the engine assembles, around that crop, the regional, soil, topographic, climatic, timing and practice evidence needed to answer it. Region remains essential knowledge, but it becomes one contributing tree among seven rather than the root of the model. This makes the same engine reusable for a farmer who does not yet know what to plant (crop discovery) and for a farmer who already has a crop and wants to improve it (crop improvement) — both are crop-centered questions.

1.3 Architectural Separation (unchanged principle)

WHAT TO ADVISE is owned by the crop knowledge forest and Developer 2. HOW TO ADVISE is implemented by Developer 1 through a generic inference engine. External data is accessed through adapters, and delivery occurs only after the canonical agricultural recommendation is built. Agricultural thresholds — including crop-specific soil, water and spacing thresholds — remain represented as rules/data, never as hard-coded if/else logic.

1.4 The Crop Passport — A Unifying, Lifecycle-Centric Concept

The merged solution is best understood as a Crop-Centric engine (Sections 1.1–1.3) combined with a Lifecycle-Centric record: the Crop Passport. When a farmer registers a target crop on a plot, the system opens a Crop Passport for that crop instance — a living record that carries the crop profile, the plot's current growth stage, its observation and recommendation history, and its planned next actions. Because the Passport is stage-aware, a farmer can enter the system at any point — pre-planting, planting, mid-growth, or post-harvest — and receive advice tailored to exactly where that plot stands, rather than starting the conversation from zero each time. In V1 the Passport is built and updated on demand, each time the farmer interacts with Mobile or receives an SMS; V2 (Section 18.2) makes it continuously and proactively refreshed from real-time data. No hardware is required for either stage — the Crop Passport becomes sensor-fed only at V3 (Section 18.3).

2. Scope, Stakeholders and Requirements

2.1 Stakeholders

Actor	Responsibility
Mobile farmer	Select/describe a target crop; ask a question; enter observations; optionally attach image evidence; receive explanation/actions; browse education content.
SMS farmer	Receive proactive, crop-calendar-based informative messages without the Flutter application.
Developer 1	Inference, evaluation, scoring, ranking, conflict resolution, recommendation generation.
Developer 2	Crop knowledge forest, agricultural rules, sources, validation, terminology.
Flutter client	Collect/display; no agricultural decision logic.
Weather provider	External current/forecast data through adapter.
Language/TTS providers	Transform canonical output after reasoning.
SMS gateway/simulator	Delivery boundary; replaceable.
Agronomy expert (V2)	Reached through expert escalation when disease/pest diagnosis needs human confirmation.
IoT soil sensor (V3)	Future automated evidence source, replacing/augmenting hand-proxy soil evidence.

2.2 Functional Requirements — V1 Crop-Centered Advisory

ID	Requirement
FR-01	Normalize agricultural context around a selected or described target crop.
FR-02	Accept a Mobile question + target crop + farmer-entered characteristics + optional image evidence.
FR-03	Generate proactive SMS advice by crop, cultivation period, region and weather, without the app.
FR-04	Use relevant past context and crop/rotation history for the target crop.
FR-05	Use present observations and current conditions relative to the target crop's requirements.
FR-06	Use future forecast, cultivation window, maturity and anticipated risks for the target crop.
FR-07	Select relevant trees dynamically, always starting from the Crop Profile tree, and deepen search when necessary.
FR-08	Retrieve and evaluate validated rules for the target crop.
FR-09	Apply constraints, score, rank and resolve conflicts between crop-specific recommendations.
FR-10	Produce a recommendation covering soil suitability, soil improvement actions, and practice advice + reasons + warnings + actions.
FR-11	Record full traceability of the crop-centered decision.
FR-12	Translate and optionally synthesize voice after reasoning.
FR-13	Format Mobile and SMS differently from one canonical result.
FR-14	Support education content in Mobile, organized by crop.
FR-15	Support weather, timing, rotation, regional risk and soil knowledge as crop-contextual evidence.
FR-16	Version and validate agricultural rules per crop.

2.3 Non-Functional Requirements

- Explainable and auditable decisions.
- Knowledge changes without rewriting inference code.
- Replaceable external providers.
- Unit, rule, integration, scenario and regression testing.
- Secure transport and controlled access to farmer context and traces.

- Graceful provider failure with explicit uncertainty.
- Extensible to richer soil information, new crops, new regions, languages and later AI/ML (see Section 18, Evolution Roadmap).

3. UML Use-Case Model

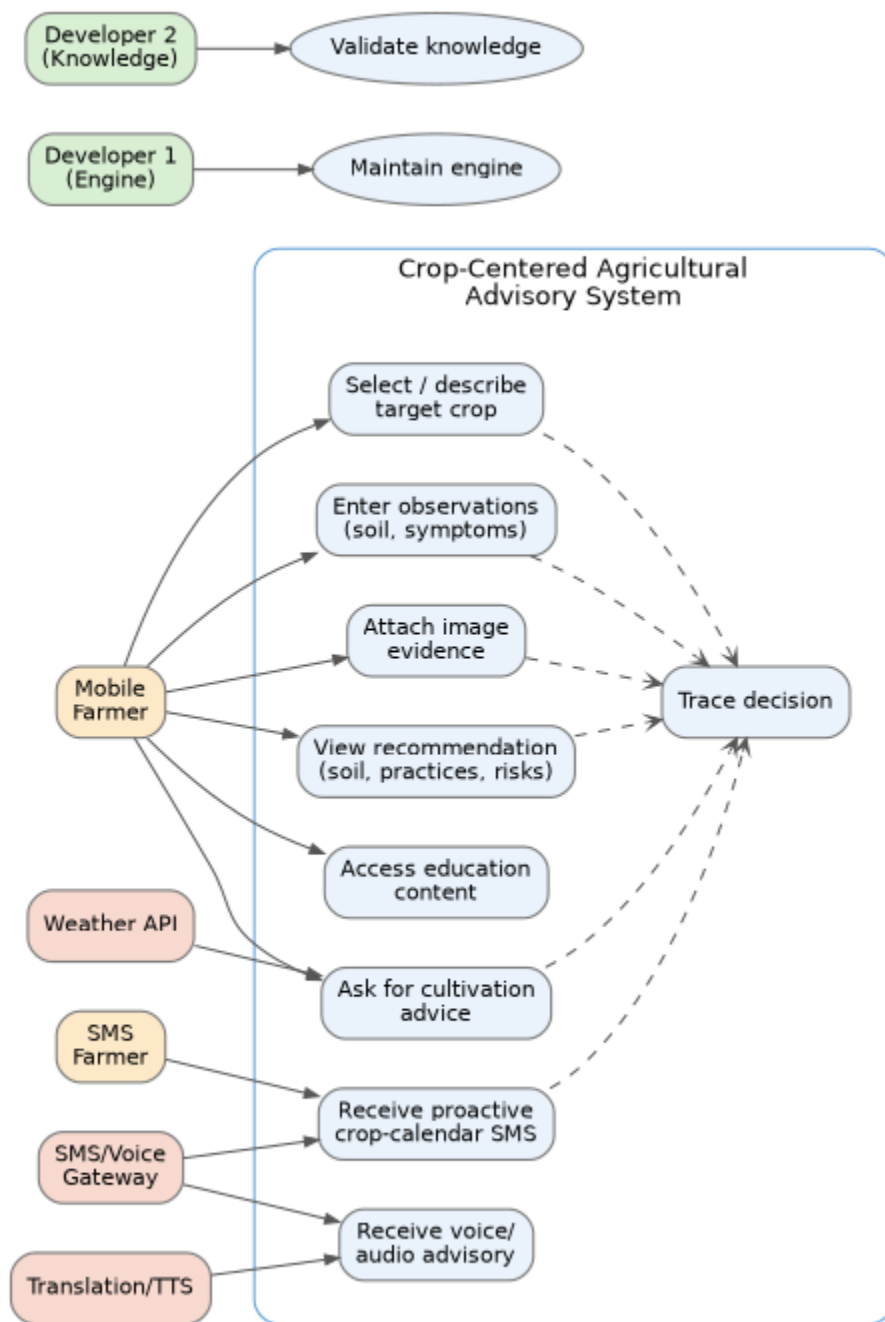


Figure 3-1 — UML Use-Case Diagram. Mobile and SMS consume the same crop-centered reasoning core; the target crop is now an explicit input the farmer selects or describes before advice is produced.

Two use cases are new relative to v2.0: “Select / describe target crop” is now the first interaction for a Mobile farmer, and “View recommendation” explicitly covers soil, practices and risk rather than a single regional answer.

4. C4 Architecture Views

4.1 C4 Level 1 — System Context

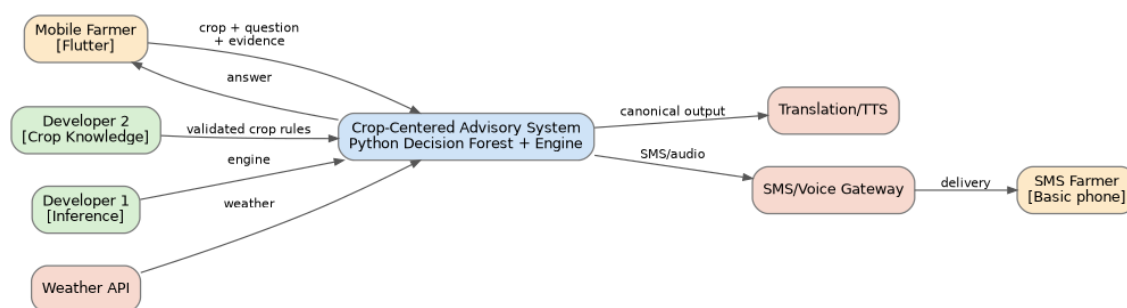


Figure 4-1 — C4 Level 1 System Context. The farmer now sends a crop together with the question and evidence.

4.2 C4 Level 2 — Container View

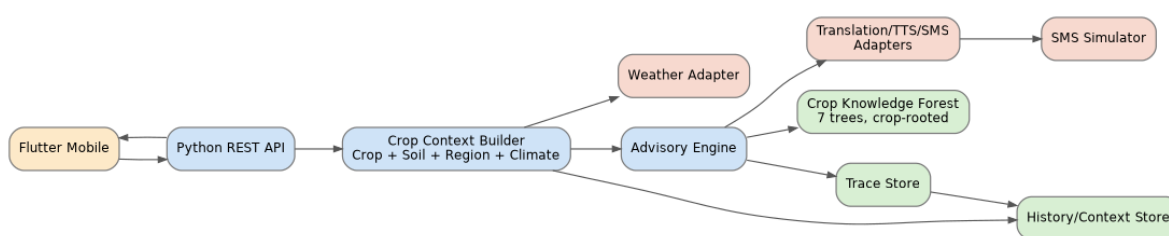


Figure 4-2 — C4 Level 2 Container View. The Context Builder is renamed Crop Context Builder: it still assembles Past + Present + Future evidence, but every piece of evidence is now gathered in reference to the selected crop.

5. Crop-Centered Knowledge Forest — Seven Working Trees

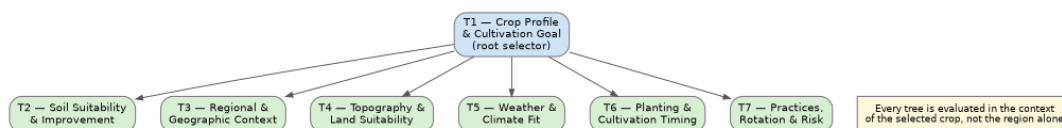


Figure 5-1 — Knowledge Forest re-rooted on the Crop Profile tree. Every other tree answers a crop-suitability or crop-improvement question, not a region-only question.

Tree	Domain	V1 content, crop-centered
T1	Crop Profile & Cultivation Goal (root)	Crop identity, growth stages, water/nutrient/spacing needs, food-security vs market goal, cycle

		length, storage behaviour. Entry point for every advisory request.
T2	Soil Suitability & Improvement	Hand-squeeze texture, water behaviour, soil colour/health proxies, evaluated against the target crop's tolerance; produces concrete improvement actions (amendment, liming, organic matter, drainage) when the soil is not yet suitable.
T3	Regional & Geographic Context	Altitude, locality and regional disease profile, used to qualify the crop's risk exposure rather than to select the crop itself.
T4	Topography & Land Suitability	Slope/hilltop, flatland/plateau, valley bottom; runoff/drainage/erosion, assessed for the target crop's rooting and water needs.
T5	Weather & Climate Fit	Regional baseline + live/forecast weather, assessed against the crop's temperature and rainfall requirements.
T6	Planting & Cultivation Timing	Three consecutive rain days, maturity before dry season, aligned to the target crop's cycle.
T7	Practices, Rotation & Risk	Rotation, restoration, mulching, disease, erosion, transport/market risk, all filtered to what is relevant for the target crop.

5.1 Worked Example — West Region Baseline, Read Crop-First

- Solanaceae crops (potato, tomato): High threat from Late Blight (*Phytophthora*) and Bacterial Wilt during heavy rainfall, July–September — surfaced only when the target crop is a Solanaceae, not for every farmer in the region.
- Coffee: Moderate/High threat from Coffee Leaf Rust (*Hemileia vastatrix*) in lower Arabica zones — surfaced only when the target crop is coffee.
- Banana/Plantain: Black Sigatoka and Bunchy Top Virus in humid valleys — surfaced only when the target crop is banana or plantain.
- Locality examples: Dschang, Bafoussam, Foumban; location still applies a regional modifier, but downstream of the crop selection.

5.2 Soil Suitability & Improvement — What Changes

Tree T2 is the tree most affected by the crop-centered redesign. In v2.0 it produced a soil description. In v3.0 it produces a suitability verdict for the selected crop plus, where the soil is not yet suitable, an ordered list of

improvement actions: for example lime or wood-ash application and organic compost for highland acid soils under Solanaceae, or drainage and raised beds for valley-bottom soils under crops intolerant of waterlogging. Improvement actions are validated agricultural rules, authored by Developer 2, never invented by the inference engine.

5.3 Soil, Climate, Timing and Practices (carried over, now crop-qualified)

- Rainy season March/April–October; dry season November–February; highland temperature baseline 15–25°C — read against each crop's own tolerance band.
- Planting confirmation checks three consecutive rain days and maturity before the November dry season, computed from the target crop's cycle length.
- Rotation blocks a repeated crop family only where the rule applies to the target crop; restoration may use Mucuna, pigeon pea or beans.
- Mulching/contour management addresses slope erosion; peak-rain disease risk and muddy-road risk can modify recommendations for the target crop.

6. Domain Model — Entities and Relationships

The domain model is presented as a data dictionary rather than a raw class diagram, for legibility. Crop is the new anchor entity: every Context, Rule and Recommendation now carries a crop reference.

Entity	Key attributes	Relationships
Farmer	farmer_id, phone, language, region, locality	submits AdvisoryRequest; holds one or more CropPassport
Crop	crop_id, name, family, cycle_length, tolerance_profile	selected by AdvisoryRequest; referenced by Rule, Context, Recommendation, CropPassport
CropPassport	passport_id, farmer_id, crop_id, plot_ref, current_stage, opened_at	living record per farmer-crop-plot instance; links stage_history[], Recommendation[] and Trace[] so the farmer can enter at any lifecycle stage
AdvisoryRequest	request_id, farmer_id, crop_id, passport_id, channel, question, objective	normalized into Context; produces Recommendation; updates CropPassport
Context	context_id, crop_id, region, soil, topography, weather, history, future	uses Rule; produces RecommendationItem
Rule	rule_id, crop_id, domain, version, priority, status, source	evaluated against Context
RuleEvaluation	evaluation_id, rule_id, outcome, evidence	recorded in Trace
Recommendation	recommendation_id, request_id, channel	contains RecommendationItem[]; recorded by Trace

RecommendationItem	item_id, candidate, score, rank	candidate action: soil improvement, practice, planting timing, etc.
Trace	trace_id, rules, constraints, scores, ranking, conflicts	recorded for every Recommendation

7. UML Component Model — Developer 1

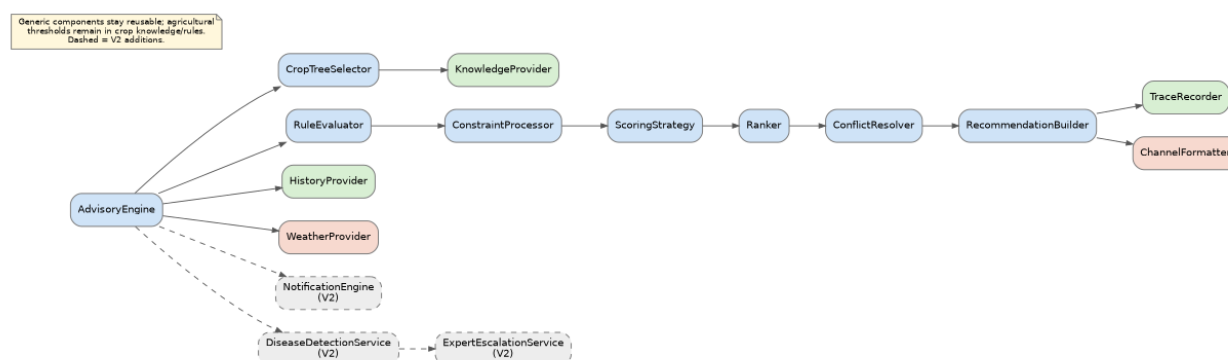


Figure 7-1 — UML Component Model. CropTreeSelector replaces the region-first selector. Dashed components (NotificationEngine, DiseaseDetectionService, ExpertEscalationService) belong to V2 and are described in Section 18; they are shown here to make the extension point explicit in V1's design.

8. Activity Model — Engine Execution

Represented as an ordered step table for legibility; the sequence is unchanged in kind from v2.0, but every step now runs in the frame of the selected crop.

#	Step	Crop-centered detail
1	Receive request	Mobile question or SMS trigger, always carrying a crop_id or a crop-discovery flag.
2	Normalize + contextualize	Builds Context around the crop: past, present, future evidence.
3	Merge relevant history	Filters farmer history to what concerns the target crop or its rotation family.
4	Select relevant trees	CropTreeSelector always includes T1 (Crop Profile); expands T2–T7 as needed.
5	Retrieve rules	KnowledgeProvider returns rules filtered by crop_id and domain.
6	Evaluate conditions	RuleEvaluator checks each rule's evidence against Context.

7	Apply constraints	ConstraintProcessor removes candidates that violate a hard, validated agronomic constraint for the crop.
8	Score candidates	ScoringStrategy computes MobileScore or SMSPriority (Section 10).
9	Rank	Ranker orders remaining candidates.
10	Resolve conflicts	ConflictResolver records evidence and rule priority when two crop recommendations disagree.
11	Build recommendation	RecommendationBuilder assembles reasons, warnings, actions, selected trees, rule references.
12	Record trace	TraceRecorder persists the full decision trail.
13	Format + deliver	ChannelFormatter renders Mobile / SMS / Voice from the one canonical result.

9. Sequence Models

9.1 Mobile Interactive Advisory

Step	From → To	Message
1	Farmer → Flutter	crop + question + evidence
2	Flutter → API	request
3	API → Context Builder	normalize (crop-first)
4	Context Builder → History / Weather	past, current/forecast
5	Context Builder → Tree Selector	trees for this crop
6	Tree Selector → Knowledge	rules for crop_id
7	Evaluator → Scorer → Ranker/Conflict	evidence → scores → ranked
8	Builder → API → Flutter → Farmer	recommendation + trace → response → answer

9.2 Proactive SMS Advisory

Step	From → To	Message
------	-----------	---------

1	Scheduler/Test Trigger → API	generate period advisory for crop cohort
2	API → Context → Weather	current/forecast for the crop's region
3	Selector → Evaluator → SMS Priority	crop-relevant rules → evaluated → prioritized
4	Builder → Formatter → SMSProvider	canonical → SMS text → delivery
5	SMSProvider → Simulator/Gateway → Basic Phone	delivery event → farmer

9.3 External Weather Failure

Step	From → To	Message
1	API → Engine → Weather Adapter	request forecast/current
2	Weather Adapter → Engine	timeout/error, or valid data
3	Engine → Fallback/Uncertainty	explicit uncertainty flag if data is missing
4	Builder → Channel	recommendation + warning (never a silently fabricated weather value)

10. Scoring, Constraints, Ranking and Conflict Resolution

10.1 Mobile Score

MobileScore(candidate, crop, context) =

crop_fit + soil_fit + soil_improvement_potential + topography_fit

+ weather_fit + regional_risk_fit + timing_fit + practice_fit

+ relevant_history_fit – risk_penalties

subject to validated hard constraints for the crop.

10.2 SMS Priority

SMSPriority(advisory, crop, context) =

weather_change_urgency + cultivation_period_relevance

+ regional_risk + soil_crop_relevance

+ practice_relevance + future_risk

subject to validated hard constraints for the crop.

10.3 Governance

The architecture defines score components, not fabricated agricultural weights. Developer 2 validates agricultural meaning and thresholds per crop; Developer 1 implements the generic scoring mechanism. A preference may increase relevance but must not override a validated hard safety/agronomic constraint for the crop.

10.4 Conflict Resolution

- Hard constraints can eliminate candidates.
- Soft factors modify ranking.
- When two recommendations conflict for the same crop, the resolver records the evidence and the rule priority used.
- If evidence is insufficient, the system produces a cautious result rather than false certainty.

10.5 Structured Output

Recommendation { crop, primary recommendation, alternatives[], reasons[], warnings[], actions[], selected_trees[], rule_references[], score_breakdown[], trace_id }

11. Interface Contracts and API Design

Interface	Representative contract	Purpose
CropProvider	get_crop_profile(crop_id)	Crop identity, tolerances, cycle.
KnowledgeProvider	get_relevant_rules(crop_id, context, trees)	Validated crop knowledge.
WeatherProvider	get_current(); get_forecast()	External weather abstraction.
HistoryProvider	get_relevant_history(crop_id)	Past context for this crop.
CropTreeSelector	select(); expand()	Dynamic forest traversal, rooted on crop.
RuleEvaluator	evaluate(rule, context)	Condition/evidence evaluation.
ConstraintProcessor	apply()	Hard/validated constraints.
ScoringStrategy	score()	Channel-specific score.
Ranker	rank()	Candidate ordering.
ConflictResolver	resolve()	Conflicting conclusions.
RecommendationBuilder	build()	Canonical recommendation.
Translator	translate()	Language transformation.
TTS	synthesize()	Audio generation.
SMSProvider	send()	Simulator/real gateway boundary.
TraceRecorder	record()/finalize()	Auditability.

11.1 API Endpoints

Method	Endpoint	Purpose
POST	/api/v1/advisory/mobile	Interactive Mobile advice for a selected crop.
POST	/api/v1/advisory/sms	Generate proactive SMS advisory for a crop cohort.
GET	/api/v1/recommendations/{id}	Retrieve recommendation + trace summary.
GET	/api/v1/crops	List available crop profiles.
GET	/api/v1/crops/{crop_id}	Retrieve a crop profile (tolerances, cycle, requirements).
POST	/api/v1/sms/simulate	Simulate SMS delivery.
GET	/api/v1/health	Health check.
GET	/api/v1/knowledge/version	Knowledge/version metadata.

11.2 Canonical Recommendation Contract

```
{ "recommendation_id": "...", "crop_id": "...", "channel": "mobile | sms | voice", "primary":
{"type":"soil_action|practice|timing|advisory","name":"...","score":0.0}, "alternatives": [], "reasons": [],
"warnings": [], "actions": [], "selected_trees": [], "rule_references": [], "score_breakdown": [], "trace_id": "..."} }
```

12. Data Architecture

The logical model follows the domain dictionary of Section 6. Crop is the new required key on Context, Rule and Recommendation; physical database technology remains an open implementation decision (Section 18.4).

12.1 Data Governance

- Knowledge rules must include crop_id, source, version and validation status.
- Advisory requests and traces must be access-controlled.
- Relevant history is used, filtered by crop; irrelevant history is not injected into decisions.
- Images are optional evidence and require controlled storage/access.
- Retention policy must be defined before production.

13. UML Deployment and Security Architecture

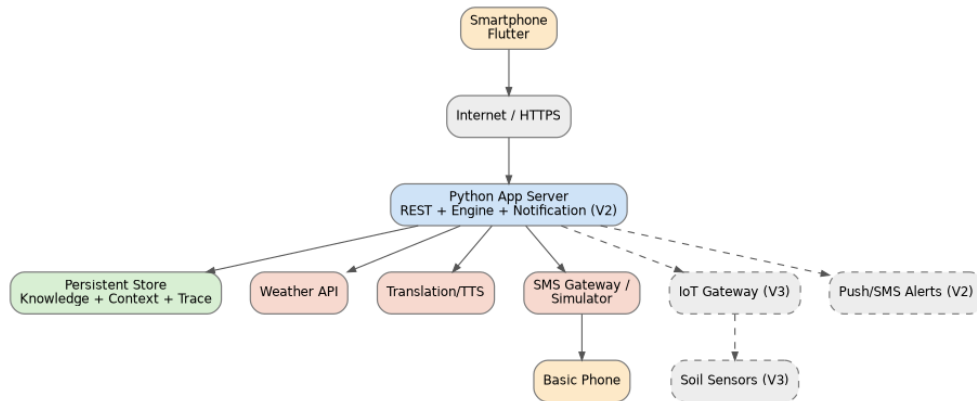


Figure 13-1 — UML Deployment Model. Solid boxes are V1 baseline; dashed boxes (IoT Gateway, Soil Sensors, Push/SMS Alerts) mark the V2/V3 extension points described in Section 18.

13.1 Security Controls

Area	Control
Transport	HTTPS/TLS.
Authentication	Exact provider/protocol open; protected operations require authentication.
Authorization	Separate farmer, internal/service and knowledge-management privileges.
Secrets	Never commit external credentials.
Input security	Validate requests, crop_id, media references, language and channel.
Trace security	Decision traces may contain farmer context; restrict access.
Provider isolation	Timeouts, validation and adapters.

13.2 Deployment Decision

V1 can be deployed as a modular monolith with strict internal interfaces. Microservices are not required to prove the architecture. The key scalability boundary is the engine/API and provider adapters, not premature infrastructure complexity.

14. Python Repository and Developer Work Split

agricultural-advisory-engine/

```

engine/
  advisory/
    engine.py
    evaluator.py
    scoring.py
    recommendation.py
  # Developer 1 — HOW TO ADVISE

```



```

selector.py          # CropTreeSelector
constraints.py
ranking.py
conflict.py
models/
interfaces/
knowledge/           # Developer 2 – WHAT TO ADVISE
crops/               # T1 – crop profiles (root)
soils/               # T2 – soil suitability + improvement rules
regional/            # T3
topography/          # T4
climate/             # T5
timing/              # T6
practices/ risks/    # T7
rules/
integrations/
weather/ translation/ speech/
sms/simulator/
languages/
api/
mobile.py
sms.py
tests/
engine/ rules/ integrations/ scenarios/

```

14.1 Responsibility Matrix

Domain	Developer 1	Developer 2	Shared
Inference engine	R	—	—
Evaluation/scoring/ranking	R	—	—
Recommendation generation	R	—	—
Crop knowledge forest	—	R	—
Crops/soils/climate/regions/seasons	—	R	—
Rotation/risks/practices	—	R	—
Integrations	—	—	R
Models/interfaces	—	—	R
Tests/scenarios	—	—	R
Final integration	—	—	R

This preserves the supplied two-engineer architecture and the repository organization defined in the v2.0 backend specification, adapted to a crop-rooted knowledge/ directory.

15. Traceability, Explainability and Past/Present/Future

```

TraceRecord { request_id, crop_id, channel, context_used, relevant_history_used, present_conditions_used,
future_conditions_used, selected_trees[], evaluated_rules[], constraints[], score_components[],
ranked_candidates[], conflicts[], final_recommendation, reasons[], warnings[], actions[], timestamps }

```

15.1 Past

- Previous relevant farmer inputs for this crop.
- Previous crops and rotation history relevant to the target crop.
- Relevant previous advisory context.

15.2 Present

- Current question and observations.
- Current soil/topography evidence, read against the target crop's tolerance.
- Current regional and weather conditions.

15.3 Future

- Forecast weather.
- Upcoming cultivation/planting window for the target crop.
- Expected maturity window.
- Anticipated disease, erosion, flood, transport or other risks for the target crop.

Traceability remains mandatory for Mobile and SMS; the system must explain selected trees, rules, conditions, constraints, scores, ranking, conflicts and final reasoning, now always anchored to a crop_id.

16. SMS Channel Architecture and Simulation

SMS remains deliberately different from Mobile. The Mobile farmer selects a crop, asks a question and supplies context. The SMS farmer primarily receives a proactive, crop-calendar-based informative message for a cultivation period or changing condition. The agricultural engine is shared; the context and scoring strategy differ, and both are now filtered by crop.

Scheduler / Test Trigger → Python API → Context Builder (crop-first) → Weather + Crop Knowledge Forest → SMS Priority Scorer → Recommendation Builder → SMS Formatter → SMSProvider Interface { V1: SMS Simulator → Virtual Inbox/test phone; Future: Real SMS Gateway → Farmer Basic Phone }

16.1 What the Simulator Needs

- A virtual recipient identifier; no IMEI is required.
- A generated message/advisory request, tied to a crop_id.
- A simulated delivery event and inbox.
- The same SMSProvider interface as production.
- No Flutter application is required for the SMS farmer.

16.2 SMS Content Priorities

- Rapidly changing weather and immediate agricultural alerts for the farmer's registered crop(s).
- Relevant cultivation period for that crop.

- Regional disease/risk conditions relevant to that crop.
- Soil and crop relevance, including improvement reminders.
- Rotation/practice reminders.
- Future risk.

17. Testing, Validation and Risk Management

Test layer	Scope
Unit	Evaluator, constraints, scoring, ranker, conflict resolver, builder.
Rule	Positive, negative and boundary cases for each validated crop rule.
Integration	Weather, translation, TTS, SMS simulator.
Scenario	Complete multi-tree, crop-first decisions.
Channel	Mobile interactive vs proactive SMS.
Trace	Every result has complete evidence.
Regression	Knowledge changes preserve validated scenarios, per crop.

17.1 Example Scenario

Input: crop = Irish potato / farmer in Bafoussam / August / heavy rainfall / flatland

Selected: T1 (potato profile) + T2 (soil) + T3 (region) + T5 (weather) + T6 (timing) + T7 (practices)

Evidence: high Solanaceae threat + July–September heavy-rain period, both retrieved because the crop is a Solanaceae

Process: evaluate → constraints → score → rank → conflict resolution → explanation

Output: recommendation (soil action + planting guidance + disease watch) + reasons + warnings + actions + trace

17.2 Risk Register

Risk	Mitigation
Wrong agricultural threshold	Developer 2 validates sources/status per crop; rule regression tests.
Conflicting conclusions	Constraints + scoring + explicit conflict resolver.
Insufficient input	Mobile follow-up; cautious result.
Weather failure	Adapter + timeout + explicit uncertainty.
SMS failure	Provider abstraction + simulator.

Language error	Canonical output + validated language resources.
Image ambiguity	Treat as evidence; avoid false certainty.
Knowledge drift	Version/source/validation metadata, per crop.
Provider lock-in	Interfaces/adapters.
Wrong crop selected by farmer	Confirmation step in Mobile; crop discovery mode for undecided farmers.
Premature AI	Rule engine remains V1–V2 baseline/fallback; AI arrives only after reliable data, at V3+.

18. Evolution Roadmap

This section is the architectural response to the product direction beyond V1: the system evolves from an on-demand advisory tool into a full cultivation assistant, then into a sensor-fed system, then into a multi-sector platform. Each stage keeps the crop-centered core, the WHAT/HOW separation, and the Mobile + SMS + Voice + education channels described in Sections 1–17; it only adds new evidence sources, new components and, at V4, new knowledge domains.

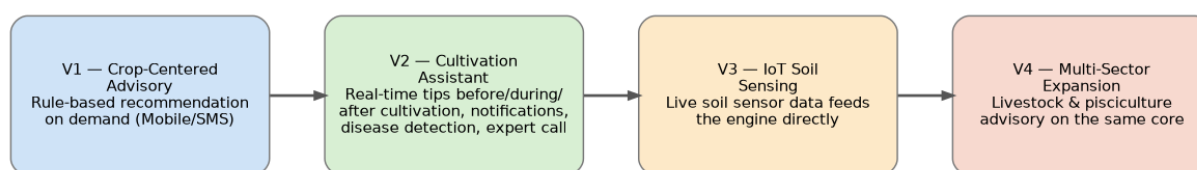


Figure 18-1 — Evolution Roadmap. Each stage is additive: nothing already delivered is removed.

18.1 V1 — Crop-Centered Advisory (this document's baseline)

On-demand, rule-based recommendation through Mobile and SMS: crop selection, soil/region/topography/climate/timing/practice evaluation, and a traceable recommendation of soil-improvement and cultivation actions. No real-time sensing, no automated notification, no image-based disease detection. This is the system specified in Sections 1–17.

18.2 V2 — Cultivation Assistant

V2 turns the system from a question-answering advisor into an assistant that accompanies the farmer through the full cultivation cycle — before, during and after — for the crop(s) they have registered. Architecturally, V2 is what happens when the Crop Passport (Section 1.4) moves from being refreshed on demand to being refreshed continuously from real-time data: the same record, read more often.

18.2.1 Lifecycle Guidance

- Before cultivation: land preparation tips, soil improvement checklist, seed/variety selection, timing confirmation — all already producible by V1's engine, now pushed proactively rather than only on request.

- During cultivation: stage-based tips (germination, vegetative growth, flowering, fruiting) driven by elapsed time since planting and by the crop's cycle model in T1.
- After cultivation: harvest timing, storage and post-harvest handling advice, and rotation guidance for the next cycle, closing the loop back into T7.

18.2.2 Real-Time Information Reliance

V2 depends on continuously refreshed inputs rather than only on-request evidence: live weather feeds (already abstracted by WeatherProvider), a cultivation calendar per registered crop and farmer, and any additional data sources the pilot phase makes available. The WeatherProvider and CropProvider interfaces defined in Section 11 are designed to absorb this without a contract change — only the refresh cadence changes, from on-request to scheduled.

18.2.3 Notification Engine

A new NotificationEngine component (introduced dashed in Figure 7-1) evaluates registered farmers' crop/region/stage combinations against incoming real-time data on a schedule, and raises notifications for:

- Disease-risk periods for the farmer's registered crop (e.g. entering the July–September Late Blight window for Solanaceae).
- Harsh or unusual climate periods (heavy rain, drought onset, temperature extremes) relative to the crop's tolerance.
- Situation-based advice keyed to what the farmer has reported (e.g. a farmer who logged a symptom last week receives a follow-up check-in).
- Any other event the crop knowledge forest marks as farmer-notifiable — the same rule/priority mechanism as SMS Priority (Section 10.2), reused rather than duplicated.

Delivery reuses the existing Mobile push channel and the SMS channel; no new delivery infrastructure is required beyond a scheduler and the notification-specific priority rules.

18.2.4 Plant Disease/Pest Detection and Response

A new DiseaseDetectionService (dashed in Figure 7-1) accepts the image evidence already supported by FR-02/Mobile in V1 and classifies it against a validated symptom catalogue for the farmer's registered crop. Its output feeds the same RecommendationBuilder as any other evidence, producing one of three outcomes:

- Self-treatable: the recommendation includes concrete, validated control steps the farmer can apply directly (e.g. affected-leaf removal, a specific organic or approved treatment, spacing/airflow correction).
- Guided monitoring: the recommendation asks the farmer to observe specific indicators over a defined period and re-submit evidence.
- Expert escalation: a new ExpertEscalationService routes the case, with its evidence and trace, to a human agronomy contact when the detection confidence is low, the risk is high, or the rule catalogue marks the condition as requiring expert confirmation — consistent with the V1 principle of a cautious result over false certainty (Section 10.4).

18.2.5 V2 Component and Data Impact

NotificationEngine, DiseaseDetectionService and ExpertEscalationService are added to the Developer 1 component set (Figure 7-1) without altering the Developer 2 knowledge forest's structure — they consume KnowledgeProvider and CropProvider exactly as the V1 engine does. TraceRecord gains a notification_id / detection_id reference so V2 decisions remain as auditable as V1's.

18.3 V3 — IoT Soil Sensor Integration

V3 replaces the V1/V2 hand-proxy soil evidence (hand-squeeze texture, visual colour) with live sensor readings where hardware is deployed, while keeping the hand-proxy path as a fallback for farmers without sensors.

- A SoilSensorProvider interface, symmetrical to WeatherProvider, is introduced; T2 (Soil Suitability & Improvement) consumes it exactly as it consumes farmer-entered evidence today — no change to the rule format, only to the evidence source.
- An IoT Gateway (Figure 13-1, dashed) aggregates readings (moisture, pH, temperature, and later NPK) from field sensors and forwards them through the same adapter-isolation principle already applied to Weather and SMS.
- Soil-improvement recommendations (Section 5.2) become measurable and verifiable: an action's effect can be checked against a later sensor reading rather than only against the farmer's next self-report.
- Sensor absence or failure follows the same graceful-degradation pattern as Section 9.3's weather failure: explicit uncertainty, never a fabricated reading.

18.4 V4 — Multi-Sector Expansion

V4 extends the platform beyond crop cultivation to two additional agricultural sectors, reusing the same generic engine and the same WHAT/HOW separation established in Section 1.3.

- Livestock husbandry: a new knowledge domain (analogous to Crop, e.g. “Animal Profile”) rooted the same way as T1, with its own suitability, health and practice trees (feeding, housing, disease, breeding cycle).
- Pisciculture (fish farming): a further knowledge domain covering pond/tank suitability, water quality, stocking density and feeding schedule, again evaluated by the same generic RuleEvaluator/ScoringStrategy/ConflictResolver pipeline.
- The AdvisoryEngine, RecommendationBuilder, TraceRecorder and channel formatters (Mobile/SMS/Voice) are sector-agnostic already by construction (Sections 7 and 11) and require no redesign; only new KnowledgeProvider content and, where useful, new sensor providers are added.
- A farmer's Mobile profile can hold a mixed portfolio — crops, animals, fish ponds — and receive consolidated notifications across all three from the same NotificationEngine introduced at V2.

18.5 Roadmap Summary Table

Stage	Adds	Depends on
V1 Crop-Centered Advisory	Crop-rooted knowledge forest; Mobile + SMS + Voice + education; on-demand recommendation	This document, Sections 1–17

V2 Cultivation Assistant	Lifecycle tips; real-time weather/calendar reliance; NotificationEngine; DiseaseDetectionService; ExpertEscalationService	V1 engine, KnowledgeProvider, CropProvider unchanged
V3 IoT Soil Sensing	SoilSensorProvider; IoT Gateway; measurable soil-improvement verification	V1 T2 rule format; V2 real-time ingestion pattern
V4 Multi-Sector Expansion	Livestock and pisciculture knowledge domains; multi-sector Mobile profile and notifications	V1 generic engine; V2 NotificationEngine

18.6 Open Decisions by Stage

Decision	Stage	Status
Python version / API framework / database engine / cloud provider	V1	Open
Authentication/RBAC	V1	Open
Real SMS provider / Translation provider / TTS provider	V1	Open
Exact score weights	V1	Requires agricultural validation, per crop
Hard/soft rule classification	V1	Per rule, pending validation
Retention/privacy policy	V1	Required before production
Scheduling/queue technology for notifications	V2	Open
Image classification model/provider for disease detection	V2	Open
Expert contact channel (call, chat, ticket)	V2	Open
Sensor hardware and communication protocol	V3	Open
Sensor procurement/pilot region	V3	Open
Livestock and pisciculture knowledge sourcing	V4	Open

19. Architectural Acceptance Baseline

- Python is the advisory engine technology.
- Flutter is the smartphone frontend.
- V1 has no IoT hardware; IoT arrives at V3.

- The decision model is an interconnected rule-based forest, not an ML random forest.
- The seven trees are re-rooted on the Crop Profile tree (T1); region, soil, topography, climate, timing and practices remain the same seven domains, now evaluated crop-first.
- Mobile accepts a target crop, a farmer question, characteristics and optional image evidence.
- SMS is proactive/informative, crop-calendar based, and does not require the Flutter application.
- Mobile and SMS share reasoning but use different context/scoring/presentation.
- Past, present and future remain mandatory reasoning dimensions.
- Every recommendation is traceable, including at V2, V3 and V4.
- External providers are isolated behind interfaces at every stage.
- Developer 1 builds HOW TO ADVISE; Developer 2 builds WHAT TO ADVISE, at every stage.
- The mobile education section and the SMS informative channel are preserved across all versions.

19.1 Source Basis and Traceability

This revision restructures the v2.0 backend and Smart Agricultural Advisory conception documents around the crop rather than the region, and adds the V2–V4 evolution roadmap (Section 18) requested for the next iterations of the project: a cultivation assistant with real-time reliance, notifications and disease detection with expert escalation (V2), IoT soil sensing (V3), and multi-sector expansion to livestock and pisciculture (V4).

19.2 Professional Deliverables Included

- UML Use-Case Diagram (crop-centered)
- C4 Context Diagram
- C4 Container Diagram
- Domain data dictionary
- UML Component Diagram (with V2 extension points)
- Activity model (step table)
- Sequence models — Mobile, SMS, failure handling
- Logical data model and data governance
- UML Deployment Diagram (with V2/V3 extension points)
- Requirements specification
- Interface contracts and API catalogue
- Scoring and conflict-resolution model
- Repository and responsibility matrix
- Security and governance
- Testing strategy and acceptance criteria
- Risk register
- Evolution roadmap V1–V4 with open decisions per stage